DATABRICKS CERTIFIED DATA ENGINEER
ADVANCED TRACK

Chapter 8

DevOps & CI/CD for Databricks Projects

Declarative Automation Bundles (DABs) • Variables & Templating • Testing Strategy
VSCode Development • CI/CD Best Practices • GitHub Actions & GitFlow

Comprehensive Exam-Ready Study Guide & Revision Notes

Compiled from official Databricks Advanced Data Engineer course material,
with added explanations, definitions, and current documentation references.

Prepared for Parth — Ahmedabad, India

How to Use This Guide

This guide rebuilds the full content of your Chapter 8 slide deck ("DevOps and CI/CD Review") into a structured revision document, in the same sequence the lecture follows. Every concept from your slides is included, with extra plain-language explanations, worked examples, and definitions of technical terms (including Terraform, IaC, YAML, GitFlow, and more) added for clarity.

- Sections 1–2 recap core DevOps/CI-CD theory and deployment tooling.
- Sections 3–6 are the exam-heavy core: Declarative Automation Bundles (DABs), the databricks.yml file, variables, and templates.
- Sections 7–9 cover applying this in practice: local development, best practices, and GitHub Actions/GitFlow automation.
- Section 10 previews Git fundamentals (the start of your next chapter).
- The Glossary defines every important term in one place — use it as a quick lookup during revision.
- The Quick Revision Cheat Sheet and Practice Questions near the end are designed for last-minute review before your exam.

> **Exam tip:** Databricks renamed “Databricks Asset Bundles (DABs)” to “Declarative Automation Bundles” in current documentation. Both names refer to the exact same feature — your slides and the exam may use either term, and this guide uses them interchangeably.

Section 1 — DevOps & CI/CD Foundations Review

1.1 What is DevOps?

DevOps is a culture and set of practices that combines Software Engineering Best Practices with IT Operations to deliver software more rapidly, efficiently, and with higher quality. It fosters collaboration between development and operations teams to automate workflows, streamline processes, and ensure continuous delivery of applications.

In short: DevOps bridges the gap between "building software" (Dev) and "running software reliably" (Ops), so that changes reach production quickly and safely.

Key benefits of DevOps

- Faster deployment cycles
- Improved collaboration between teams
- Enhanced system reliability
- Better scalability and efficiency

The DevOps lifecycle

DevOps is a continuous, cyclical process, not a one-time project. It is usually visualized as an infinity loop with two connected halves:

- Dev half: Plan → Code → Build → Test
- Ops half: Release → Deploy → Operate → Monitor

The cycle repeats continuously — feedback from monitoring production feeds back into planning the next set of fixes, updates, or features.

1.2 DataOps and MLOps — DevOps Applied to Data & ML

Once you understand DevOps generally, two specialized applications of it matter for data engineers:

- **DataOps** is DevOps applied to Data Engineering. It automates the management of data pipelines, ensuring smooth, reliable data flows from collection through processing, with fewer bottlenecks and faster insights. Think of it as: automation + collaboration + testing, but applied specifically to your ETL/ELT pipelines.
- **MLOps** is DevOps applied to Machine Learning. It streamlines deploying and managing ML models, ensuring models move from development to production quickly and are continuously monitored for performance (drift, accuracy, etc.).

> **Exam tip:** A simple exam distinction: DataOps = pipelines/data quality; MLOps = models/experiments. Both sit "under" the DevOps umbrella and use the same CI/CD principles.

1.3 CI/CD Fundamentals

CI/CD is the concrete, tool-driven implementation of DevOps principles. It is a subset of DevOps practices that focuses on automating code integration, testing, and delivery.

Continuous Integration (CI)

CI involves regularly merging code changes from multiple contributors into a central repository and running automated tests to ensure code quality. Instead of developers working in isolation for weeks, they integrate constantly — which surfaces conflicts and bugs early.

- Key practices: frequent commits (several times a day), automated build and test on each commit, fast feedback on errors.
- Typical flow: developer commit → build server → test → feedback loop.

Benefits of Continuous Integration

- Early detection of issues — integrating code often means bugs and conflicts are caught early, making them easier to fix.
- Faster development cycle — frequent integration speeds up delivery of new features and fixes.
- Improved collaboration and code quality — regular, modular code and better teamwork.
- Automated testing and validation — automated tests run with each integration to ensure the code is stable and works with existing features.

Continuous Delivery vs. Continuous Deployment (both abbreviated "CD")

These two terms are frequently confused but mean different things — a very common exam trap:

Term	What happens	Human involvement
Continuous Delivery	Automatically pushes changes to staging / pre-production environments, with the ability to manually deploy to production at any time.	Manual approval / manual trigger required before production release.
Continuous Deployment	A fully automated process where each change passing all tests is immediately deployed to production.	No manual intervention — fully automatic.

High-level workflow both share: Develop → Build → Test → Version Control → Deploy to Stage → Deploy to Production. Continuous Delivery stops just short of the final production push (a human clicks "deploy"); Continuous Deployment completes that last step automatically.

> **Exam tip:** If a question asks "which CD requires manual approval to reach production?" — the answer is Continuous Delivery, not Continuous Deployment.

The Testing Pyramid

CI/CD relies on a layered testing strategy known as the testing pyramid, which categorizes tests by scope, speed, and cost:

Layer	Speed / Cost	What it tests	Example on Databricks
Unit Tests	Fast, low cost, high coverage, automated (base of pyramid)	Individual functions or methods in isolation	Testing a custom PySpark function
Integration Tests	Slower, more costly than unit tests	The interaction between different components or systems	Testing whether a notebook / Lakeflow

Layer	Speed / Cost	What it tests	Example on Databricks
			Declarative Pipeline / Job works correctly together
System Tests	Slow, costly, run less often (top of pyramid)	The entire application, end-to-end, in a real-world scenario	Testing an end-to-end data pipeline inside a Workflow

Unit tests form the foundation because they are inexpensive and provide the broadest coverage; system tests sit at the top because they are the slowest and most expensive, but give the highest confidence that the whole pipeline works as expected in production-like conditions.

1.4 Isolating Environments for CI/CD

Isolating environments for the different stages of development is key to a solid CI/CD pipeline. This ensures code is developed and tested away from production, minimizing risk.

- Workspaces: utilizing multiple Databricks workspaces, one for each environment (Dev, Stage, Prod).
- Catalogs: utilizing multiple Unity Catalog catalogs, one for each environment, even within a shared workspace.

Minimal setup requires at least two environments — "Development & Stage" and "Production" — but this can vary depending on your organization's requirements.

Setting Up Your Data for CI/CD

Environment	Data characteristics
Dev Data	• Often a small static subset of production data • Can be anonymized or use synthetic datasets • Supports rapid development and testing • Ensures privacy and data integrity
Stage Data	• Staging data mirrors production structure and volume (typically static) • Can be anonymized or scrubbed of sensitive information • Ensures realistic testing and validation
Prod Data	• Production data: live & fully operational • Contains real user data, continuously updated • Requires high security, privacy, and compliance standards

> **Exam tip:** Each environment must have a data setup suited to its purpose — balancing realism, security, and sensitivity at every stage. This is a recurring exam theme: dev = small/synthetic, stage = structurally realistic but scrubbed, prod = live and fully governed.

Section 2 — Deploying Databricks Projects

2.1 Deployment Tools Overview

There are several tools available to deploy your Databricks projects. Each trades off ease-of-use against control and power:

Tool	How it works	Notes
REST API	Provides direct access to Databricks functionality through HTTP requests.	Requires manual construction of HTTP requests and handling the returned responses. Most low-level, most flexible.
Databricks SDK	SDKs for Python, Java, Go, and R that help accelerate development and deployment.	Covers all public Databricks REST API operations and supports multiple programming languages.
Databricks CLI	An easy-to-use interface for automating tasks from the terminal, command prompt, or bash scripts.	Uses Declarative Automation Bundles (DABs) to write infrastructure as code for Databricks. → This is the method used throughout this course.

2.2 Using the Databricks CLI — Connection & Authentication

There are a variety of ways you can run Databricks CLI commands, depending on your workflow:

Method	Details
Web Terminal	• Run Databricks CLI commands directly from the Databricks web terminal • Enables you to run shell commands within Databricks • Uses the latest version of the Databricks CLI by default • Authentication is based on the current user; must be enabled in your environment
VS Code	• Install and use VS Code to install and use the Databricks CLI • You must authenticate to Databricks • The Databricks VS Code Extension provides additional features
Databricks Notebook	• Run shell commands with %sh in a Databricks notebook • Install and use the Databricks CLI directly inside a notebook • You must authenticate to use the CLI via a token • Main method used in this course (simple and easy for learning — in production, a different approach such as CI/CD automation may be preferred)

2.3 Typical Databricks Project Components

A typical Databricks project consists of three broad areas:

- **Code:** Notebooks, Python/YAML/JAR files, DBT, etc.
- **Execution environment:** Databricks workspaces and compute configuration.
- **Resources:** Databricks workflows/jobs, MLflow, Lakeflow Declarative Pipelines, Lakeflow Jobs, and more — these produce a variety of data products, such as tables, dashboards, machine learning models, and call-external services.

Deliverables that these components produce range from a simple report (one or more notebooks running on single-node compute) to a full MLOps pipeline (requiring MLflow, Feature Store, and Model Serving components together).

2.4 The CI/CD Journey to Production

At a high level, the journey follows a version-control-driven path: work begins on a feature branch, moves through development and staging, and is finally promoted to production.

- From development: begin by deploying your Databricks project to the development environment. This allows you to test changes and iterate as you make progress on development data using specific development configurations.
- Development → staging: after completing development and creating a pull request into the staging branch, this triggers automated testing of your project. This ensures continued testing on new staging data with the appropriate staging configurations.
- Staging → production: once both development and staging environments have completed testing and passed all tests, you can deploy to your main / production branch. This fully deploys your project to production, utilizing production data and the necessary production configurations.

Example environment-specific compute configuration

Environment	Example configuration
Development	• Single-node compute • Small subset of data, isn't a large scale • Data used is user account-based
Staging	• Serverless compute to closely resemble the production environment • Using a service principal identity, which grants automated deployments better security than using an individual's account
Production	• Continue using Serverless compute • The project runs on production data • Set to run on a weekly / defined schedule to keep data updated for consumers

2.5 How Do We Orchestrate This CI/CD Journey?

There are three broad ways to orchestrate deployment across environments — each with distinct tradeoffs:

Method	Pros	Cons
Manually (using the UI)	Easy to learn, high level.	Time-consuming; error-prone; not a viable option for CI/CD (requires human interaction, so it is highly error-prone and not scalable).
Programmatically (Databricks REST API or SDK)	Low-level control.	Requires deeper programming knowledge — many API calls or SDK classes are needed to automate an entire CI/CD process, which can be extremely time-consuming.
Terraform (Databricks Terraform provider)	Very powerful and expressive tool of choice for administrators to manage infrastructure.	Can be challenging to learn for data scientists and engineers without a deep infrastructure-management background — it adds another tool you have to learn and manage.

Exam tip: This slide is where Terraform is introduced in your course — as one of three ways to orchestrate CI/CD deployment. See the Glossary for a full explanation of what Terraform is and how it relates to Databricks.

Section 3 — Declarative Automation Bundles (DABs) Deep Dive

3.1 How Can the CI/CD Process Be Simplified?

Before introducing DABs, it helps to see the problem they solve. What if we could:

- Co-version code with all configurations in a simple, easy-to-understand format like YAML?
- Define Databricks resources using existing REST API parameters?
- Ensure user isolation during deployment?
- Specify environment-based overrides and variables?

YAML is a human-readable format that organizes data in a simple, easy-to-read structure using indentation and minimal syntax — much easier to understand and edit compared to other formats like XML or JSON.

3.2 What Are Declarative Automation Bundles (DABs)?

Declarative Automation Bundles (formerly called Databricks Asset Bundles) are a tool to facilitate the adoption of software engineering best practices, including source control, code review, testing, and continuous integration and delivery (CI/CD), for your data and AI projects.

DABs use YAML files to specify the artifacts, resources, and configurations of a Databricks project. The core idea is: write code once, deploy everywhere. Moving forward, Databricks recommends DABs for creating, developing, deploying, and testing jobs and other Databricks resources as source code.

How do DABs work?

The Databricks CLI has specific bundle commands to validate, deploy, and run Declarative Automation Bundles using a bundle YAML file (databricks.yml).

Where are DABs used?

Bundles are extremely useful during development and CI/CD processes to deploy Databricks assets to specified environments, while modifying specific configurations for each environment.

3.3 CI/CD With DABs — Overview

A typical bundle-driven development and CI/CD pipeline looks like this:

- Local development: build the project bundle with your team, using a local development environment. You can either edit bundles manually or develop within a Databricks workspace, testing changes away from production.
- After development: users can manually deploy and test changes away from production — committing changes to version control and pushing your development changes to a project repository.
- After changes are committed: set a notification to trigger the CI/CD pipeline, testing the first deployment to a staging environment.
- Once all tests pass: the code can be deployed into the production environment after following your organization's processes, such as code review and approval.

3.4 Simple Project Structure Overview

A typical Databricks Asset Bundle project folder looks like this:

```
my_project/
├── resources/      # Additional YAML configuration files for your DABs
├── src/            # Contains the source files needed for the data pipeline
│                (notebooks, python files, etc.)
├── tests/          # Contains unit and integration tests for the pipeline
└── databricks.yml  # REQUIRED bundle configuration file
```

- **resources/** should contain any additional YAML configuration files for your DABs, if needed.
- **src/** (or source folder) contains the source files needed for the data pipeline, such as notebooks and Python files.
- **tests/** contains your unit and integration tests for the data pipeline.

The databricks.yml file is a required bundle configuration file used to deploy Databricks assets. This file must:

- Be expressed in YAML format.
- Contain at minimum the top-level bundle mapping.
- Contain at least one (and only one) bundle configuration file named databricks.yml (additional files can be pulled in via the include mapping).

3.5 databricks.yml — Top-Level Mappings

The databricks.yml file is key to your bundle. In a simple example, it includes the following top-level mappings: bundle, resources, and targets. Other top-level mappings include variables, workspace, permissions, artifacts, include, and sync. We'll cover many of these throughout this course.

Top-level mapping	Purpose
bundle	Required. Declares the bundle's name and can also set other optional configuration, such as a required Databricks CLI version.
resources	Declares the Databricks resources used by the bundle — Lakeflow Jobs, Lakeflow Declarative Pipelines, MLflow, and more — defined using the corresponding Databricks REST API parameters.
targets	Specifies deployment environments and environment-specific configuration overrides (for example: development vs. production).
variables	Declares custom variables (simple or complex) that can be referenced and overridden throughout the bundle configuration.
workspace	Specifies the Databricks workspace-related settings, such as the host URL, used for the bundle.
permissions	Defines the permissions to apply to resources defined in the bundle.
artifacts	Specifies build settings for artifacts (for example, Python wheels) that are automatically built during bundle deployment.
include	Specifies a list of path globs that contain additional configuration files to include within the bundle (beyond databricks.yml itself).
sync	Specifies local paths to synchronize to the workspace — useful for sharing common files across bundles.

Example databricks.yml walkthrough

Here is a simple bundle configuration file that ties several of these mappings together:

```
bundle:
  name: demo01_bundle

resources:
  jobs:
    l1_simple_dab:
      name: my_job_name_l1_simple_dab
      tasks:
        - task_key: create_bronze_table
          notebook_task:
            notebook_path: ./src/create_bronze_table.py
            source: WORKSPACE

targets:
  development:
    mode: development
    default: true
    workspace:
      host: https://<dev-workspace>.cloud.databricks.com/
  production:
    mode: production
    workspace:
      host: https://<prod-workspace>.cloud.databricks.com/
```

Reading this configuration:

- A bundle configuration file must contain only one top-level bundle mapping that associates the entire bundle's content with a name. Additionally, you can include other Databricks workspace settings, such as cluster_id, compute_id, git, or others; the additional mappings beneath the top-level bundle mapping are indented.
- **resources** declares information about the Databricks resources used by the bundle — Lakeflow Jobs, Lakeflow Declarative Pipelines, MLflow, and more — defined using the corresponding Databricks REST API parameters.
- Each resource (here, a job) is created using the name mapping. This job creates a job named my_job_name_l1_simple_dab using the name mapping, and this job contains one or more tasks specified using the corresponding REST API parameters.
- Within the tasks mapping, the task uses notebook_task and uses the notebook_path key to specify the notebook to use. Be aware: starting December 20, 2024, the default format for new notebooks will be .ipynb format. If you do not supply the correct notebook extension, an error will be returned (in this example, the notebook is using the traditional .py extension).
- The top-level mapping targets sets specific environments and environment-related configuration, including environment mode type, whether it should be set as the default target when you run and if it should be inherited to the development target, along with various other configurations and overrides. In this example, we include two target environments: development and production, each with unique configurations and overrides. For development, we specify the mode: development mapping and a default: true mapping. For production, we specify mode: production and the production workspace URL.

3.6 Validate, Deploy, and Run Your DAB (Using the Databricks CLI)

Once you have a databricks.yml file, here's how to work with it via the Databricks CLI:

Step	Command	What it does
1. Validate	databricks bundle validate	Reviews required resource configurations for any accuracy issues in your bundle.
2. Deploy	databricks bundle deploy -t development	Specifies which environment your bundle should be deployed to. By default, deployment is set as the default target, so if you don't specify where to deploy, it will deploy to the default target; however, it's best practice to be explicit.
3. Run	databricks bundle run -t development l1_simple_dab	Runs your bundle in the development environment. You must run the job with its resource key here (the key is l1_simple_dab).

Exam tip: Command pattern to remember for the exam: databricks bundle validate → databricks bundle deploy -t <target> → databricks bundle run -t <target> <resource_key>. Also remember: the resource key (e.g. l1_simple_dab), not the display name, is what you pass to bundle run.

Section 4 — Variable Substitutions in DABs

Declarative Automation Bundles support substitutions and custom variables, making your bundle configuration files more modular and reusable. Both substitutions and custom variables allow dynamic retrieval of values, meaning that settings can be determined at the time the bundle is deployed and run.

To use a variable in your YAML file, reference it using a dollar sign, curly braces, and the mapping, then the variable name: ${...}

4.1 Default Substitutions

By default, there are a variety of variable substitutions available, including:

Substitution	Resolves to
${bundle.name}	The name of the bundle
${bundle.target}	The name of the target the bundle is currently deployed to
${workspace.file_path}	The workspace file path for the bundle
${workspace.root_path}	The workspace root path for the bundle
${resources.jobs.<job-name>.id}	The ID of the named job resource
${resources.models.<model-name>.name}	The name of a named model resource
${resources.pipelines.<pipeline-name>.name}	The name of a named pipeline resource

Common ones to remember include bundle.target for the value of the target environment, and workspace.file_path for the workspace file path, among others.

4.2 Simple Custom Variables

You can also define simple custom variables in your bundle to enable dynamic retrieval of values needed for various scenarios. Custom variables are declared in your bundle configuration files within the variables mapping.

By default, a simple custom variable is assumed to be of type string. Example: create a variable named my_lab_user_name, along with a description and its default value.

```
# databricks.yml example
variables:
  my_lab_user_name:
    description: Your user name
    default: labuser23904

catalog:
  catalog_dev:
    description: Development catalog reference
    default: ${var.my_lab_user_name}_1_dev
  catalog_prod:
    description: Production catalog reference
    default: ${var.my_lab_user_name}_3_prod
```

You can then reference the custom variable my_lab_user_name within other variables or mappings. In this example, we create catalog_dev and catalog_prod variables that reference the custom variable by appending _1_dev and _3_prod. To reference a custom variable, use the top-level variables mapping var, followed by a dot and then the variable name (var.my_lab_user_name), and then append the literal string. As a result, catalog_dev resolves to labuser23904_1_dev, and catalog_prod resolves to labuser23904_3_prod.

4.3 Complex Variables

By default, a variable is assumed to be of type string unless you define it as a complex variable. If you need more complex data, you can define a variable as a complex type.

```
# databricks.yml example
bundle:
  name: demo03_bundle

variables:
  my_cluster:
    description: "My cluster"
    type: complex
    default:
      spark_version: "15.4.x-scala2.11"
      node_type_id: "Standard_DS3_v2"
      num_workers: 2
```

In this example, we create a complex variable named my_cluster, with a default value specifying the Spark version, node type, and number of workers. This lets you provide structured, multi-field data for a single variable, instead of a plain string.

4.4 Lookup Variables — Dynamically Retrieve an Object's Value

For specific object types, you can define a lookup for your custom variable to retrieve the object's ID.

```
variables:
  my_cluster_id:
    description: "Get the cluster ID using a lookup variable"
    lookup:
      cluster: myclustername
```

First, specify the variable name (my_cluster_id). Then, use the lookup mapping to define what to look up — in this case, we are looking up the cluster name myclustername to retrieve the corresponding cluster ID value. If a lookup is defined for a variable, the ID of the object with the specified name is used as the value of the variable, ensuring the correct resolved ID of the object is always used.

Object types supported for lookup variables

At the time of this recording, lookup variables can be used for the following object types:

- alert
- cluster_policy
- cluster
- dashboard

- instance_pool
- job
- metastore
- notification_destination
- pipeline
- query
- service_principal
- warehouse

4.5 Target Environment Variable Overrides

You can dynamically modify variable values within the target top-level mapping.

```
targets:
  development:
    variables:
      target_catalog: ${var.catalog_dev}
  production:
    variables:
      target_catalog: ${var.catalog_prod}
```

For example, let's say we have a custom variable named target_catalog that is used to populate a job parameter and read from a specific catalog. To dynamically override the target_catalog variable for each environment, deploying to development uses the target_catalog variable to use the catalog_dev value; deploying to production, the target_catalog variable will use the catalog_prod value.

> **Exam tip:** A default value for the variable must be defined in the variables top-level mapping in order for an override to work as expected — if you don't specify an override for your variable, it will default to its original value.

4.6 Benefits of Using Variables in DABs

- **Customizable for different environments:** Easily modify configurations, database connections, file paths, and other settings for development, staging, and production environments.
- **Reusability across Databricks projects:** You can use the same asset bundle across multiple teams or workspaces by adjusting only the variable values, promoting consistency and reducing redundancy.
- **Easy maintenance and updates:** Quickly update assets by modifying variables to ensure consistency across environments and reduce the risk of errors.

Section 5 — DAB Project Templates

5.1 Default Bundle Templates

You have two main options here: you can either use the pre-configured templates provided by Databricks, or you can create your own custom templates to meet specific needs.

To get started with one of these templates, you can use the databricks bundle init <template_name> CLI command. This will automatically initialize your bundle based on the selected template, saving you time in the configuration process.

Template	Use case
default-python	For Python-based projects.
default-sql	For SQL-based projects.
dbt-sql	For projects involving DBT (Data Build Tool).
mlops-stacks	For machine learning operations stacks.

Exam tip: One note: when using a bundle template within a Databricks notebook, this will automatically initialize your entire template. If you run the CLI command outside of a notebook, a series of prompts allows you to select specific portions of the template.

5.2 Custom Bundle Templates

In addition to Databricks' default templates, you can create custom bundle templates specific to your organization. Although the fine detail of building custom templates is beyond this course, a few key facts are worth knowing:

- Custom templates require at least two files: databricks_template_schema.json and databricks.yml.tmpl.
- You can include user prompts, define folder structure, and customize settings.
- To use a custom template, simply pass the local path or remote URL to the Databricks CLI bundle init command.

```
databricks bundle init
  /sys/en/templates/test-template
```

Custom templates help organizations create and manage bundles in a consistent, repeatable way, establishing folder structures, tasks, and DevOps infrastructure-as-code (IaC) conventions for development and deployment.

Section 6 — CI/CD Project Overview with DABs

This section walks through how to implement a CI/CD workflow using Declarative Automation Bundles — covering how to plan a project, set up testing strategies, and deploy workloads efficiently across different environments.

6.1 Planning the Project

Declarative Automation Bundles simplify this process by allowing you to define, version, and deploy assets in a structured way using the Databricks CLI. When planning a project, it's crucial to define three things:

- **Deliverables** → What are we building?
- **Tasks** → What are the key steps to achieve it?
- **Databricks Assets** → What components are required?

Worked example used in the course: visualizing health data.

- Deliverable: Visualize health data.

1. Ingesting a daily incremental CSV file into a bronze table.
2. Cleaning the data and moving it to a silver table.
3. Creating gold tables to share with consumers.

- Databricks Assets required: Notebooks, Delta Live Tables (Lakeflow Declarative Pipelines), Workflows, Compute.

6.2 Catalog Isolation Per Environment

Now that we have an understanding of our project, our data, and how to isolate environments, let's dive into the project architecture. In this project, we create a catalog for each environment: dev, stage, and prod.

- **Dev catalog:** small subset of production data, used for initial and rapid development / testing.
- **Stage catalog:** larger subset of prod, used for comprehensive testing as we approach CI/CD.
- **Prod catalog:** full, live production data that we rely on for production operations.

Our workflow starts by developing on the dev catalog, running unit and integration tests. The pipeline is then set up so as we progress through each stage — dev, stage, prod — we ensure everything is working correctly before deploying to production.

6.3 Reviewing the Role of CI/CD Testing (Recap)

Within CI/CD there are testing steps you should be following with what's called the testing pyramid. The testing pyramid categorizes different tests: unit tests, integration tests, and system tests. Testing your code is extremely important.

- Unit tests: the base of the pyramid — test small individual functions or methods in isolation. Since they test small individual functions, they typically can run quickly, frequently, and automatically, ensuring that the functions work as expected.
- Integration tests: test the interaction between different components or systems. These are typically slower and more costly than unit tests, but provide greater assurance that components work together correctly. Within Databricks, these typically revolve around using Notebooks, Lakeflow Declarative Pipelines, or Lakeflow Jobs — for example, testing whether a PySpark method and a Declarative Pipeline work together correctly.

- System tests: lastly, system tests test the entire application, ensuring that all parts function together in a real-world scenario. These are typically slow, costly, and often run in a production-like environment. For example, for an end-to-end data pipeline, testing whether the data pipeline works as expected within a Workflow, creating our desired results.

6.4 Unit Testing with pytest

pytest is a popular testing framework for Python that makes it easy to write simple and scalable test cases.

Feature	What it means
Simple syntax	Minimal setup — define functions starting with test_.
Provides assert statements	Simple to use for detailed error messages, no need to manually check results or write your own assertion logic.
Automatic discovery	Finds and runs all tests automatically with simple configuration.
Rich ecosystem	Extend functionality with plugins for coverage, parallel testing, and more.

First, pytest has a very simple syntax. You don't need to worry about complex setup — just write test functions starting with test_, and pytest will automatically pick them up and run them.

Next, pytest leverages Python's built-in assert statements (or you can simple use assert like in PySpark) so you can quickly check for correctness in your code without needing to write manual debugging messages when something goes wrong.

pytest also automatically discovers and runs all your tests. There's no need to manually configure which tests to run — just name your test files and functions with the test_ prefix, and pytest will pick them up.

Lastly, pytest has a rich ecosystem of plugins, allowing you to extend its functionality. Whether you need test coverage reports, parallel test execution, or integration with other tools, there's a plugin to fit almost any need. This course provides a simple introduction to pytest — there are many testing frameworks available; select the one that best meets your organization's needs.

6.5 Pipeline Expectations for Integration Tests

As part of your integration testing strategy, Lakeflow Declarative Pipelines support built-in data quality expectations — constraints applied to the data flowing through a pipeline. Expectations let you validate assumptions between pipeline stages (for example, between a bronze and silver table) as part of your automated integration tests, catching data-quality problems before they propagate downstream.

Reference notebook example shared in this lecture (Databricks-hosted) — https://dbc-d1e23f24-c95a.cloud.databricks.com/editor/notebooks/634209448007576?o=3434287344179690&command=634209448007577

6.6 CI/CD With DABs — High-Level Workflow Overview

Looking at the high-level overview of CI/CD, we want to combine continuous integration (develop, build, test, and version control) with continuous delivery/deployment (deploying the project to stage and production). Automating this entire process helps us develop quickly and error free.

In the end, the CI/CD process streamlines development of your data pipelines by automating testing and deployment, leading to faster, more reliable releases. This approach minimizes manual errors, improves collaboration, and ensures high-quality software delivered quickly and consistently.

The bundle deploy command targets a specific environment using the -t flag, mirroring the promotion path from development through staging to production:

```
databricks bundle deploy -t development
databricks bundle deploy -t staging
databricks bundle deploy -t production
```

Section 7 — Developing Locally with Visual Studio Code (VSCode)

For many developers, using an IDE like VSCode is the preferred choice. With VSCode, you can interact with your Databricks environment in many ways, allowing you to stay within the IDE and avoid switching to the Databricks browser UI.

Tool	What it's for	Key points
Databricks CLI	Ideal for scripting & lightweight command-line tasks	• Useful in development and CI/CD processes • Supports unified authentication (OAuth, PATs, etc.) • More interactive debugging compared to Notebooks
Databricks Connect v2	Runs Apache Spark code remotely on a Databricks cluster from a local environment	• Ideal for interactive debugging and remote Spark job execution • Easy setup with pip install databricks-connect==<your version>
Databricks VSCode Extension	Integrates Databricks into VS Code for batch and interactive development	• Simplifies setup with resource explorers and the Databricks CLI • Supports running and debugging Python files on Databricks clusters • Keeps developers within VS Code, and provides DAB features

You can install and use Databricks Connect v2 to run Apache Spark code remotely on a Databricks cluster from a local environment, which is ideal for interactive debugging and remote Spark job execution, with easy setup via pip install databricks-connect==<your version>.

Lastly, the Databricks VSCode Extension enhances your IDE experience by integrating Databricks functionalities directly into Visual Studio Code. With the extension, you can integrate Databricks into VS Code for both batch and interactive development, simplify setup with resource explorers and the Databricks CLI, run and debug Python files directly on Databricks clusters, and stay within VS Code while accessing additional bundle user interface features.

The extension has a simple setup from the VS Code Marketplace, allowing you to get connected to compute in minutes. It provides a native experience for developers who prefer using VS Code, and enables you to execute batch workloads or start interactively debugging directly from your IDE.

Section 8 — CI/CD Best Practices for Data Engineering

When it comes to implementing CI/CD strategies in an organization, especially in data engineering, there are a few best practices to think about.

8.1 Define a Clear Testing Strategy

Establish and automate unit, integration, and end-to-end tests. Automating these tests ensures that your code functions correctly and that data integrity is maintained before deployment. This reduces the risk of errors and improves the overall quality of your codebase.

8.2 Implement a Version Control Strategy

Use a version control system using systems like GitHub to manage your code, notebooks, and configuration files. A good version control strategy is essential for collaboration across teams. I recommend clearly defining different stages of development, making it easier to track changes and resolve conflicts.

8.3 Establish a Code Review Process

It's also crucial to establish a robust code review process. Peer reviews help ensure that changes are thoroughly vetted before they are merged. Additionally, CI tools can automatically build pipelines on pull requests, offering a quick way to validate changes and catch issues early.

8.4 Automate Deployment with DABs

Next, leverage Declarative Automation Bundles (DABs) for consistent, repeatable, and automated deployments in Databricks. Automating the deployment process minimizes human error and speeds up the deployment cycle. DABs ensure that the right environment is always deployed to the correct environment, maintaining consistency across your projects.

8.5 Monitor and Optimize CI/CD Pipelines

Finally, continuously monitor the performance of your CI/CD pipelines. Optimizing for speed, reliability, and scalability ensures your pipeline runs as efficiently as possible. Gather feedback from developers, identify bottlenecks, and work toward improving automation efficiency.

By implementing these practices, you'll create a streamlined, reliable, and scalable CI/CD process within your Databricks environment. This will not only improve your development cycles but also increase the overall quality of your data-driven products.

> **Exam tip:** Summary statement from the slides: implementing CI/CD requires alignment across teams and the establishment of robust processes that ensure continuous improvement and quality.

For more information: Databricks Documentation – Best Practices for Operational Excellence — https://docs.databricks.com/aws/en/lakehouse-architecture/operational-excellence/best-practices

Section 9 — Next Steps: Automated Deployment with GitHub Actions

This section walks through a high-level overview of how to integrate GitFlow (an alternative Git branching model that involves the use of feature branches and multiple primary branches) with Declarative Automation Bundles (DABs). This workflow streamlines the process of developing, testing, and deploying Databricks projects by leveraging GitFlow's structured branching model and GitHub Actions for automation.

9.1 Creation of Feature Branches

The process begins on the very left-hand side with the developer creating a feature branch from the dev branch. This is where individual features or bug fixes are developed in isolation. Developers can deploy code directly to the DEV Databricks workspace using the databricks-cli.

9.2 Pull Request Workflow

Once the development on a feature branch is complete, a pull request is opened. After the code is peer-reviewed, it is merged into the develop branch.

9.3 Deployment to QA and Production

After merging into develop, the next step involves triggering the GH Actions workflow for Drafting a release. This deploys the changes to the QA environment using DABs, runs tests, code coverage checks and security scans, and finally creates the versioned release branch. It also creates a PR for the release branch to be merged into main.

After the Release PR is closed, it triggers another GitHub Actions workflow that deploys the bundle to PROD, creates a Release Tag, and eventually a PR for main branch to be merged into DEV.

9.4 Release Branching and Hotfixes

Finally, if a critical issue arises in production, we handle it with a hotfix branch, which is branched from main and then merged back into both main and dev after the fix is applied. This ensures that our production environment remains stable while allowing ongoing development to continue.

9.5 GitFlow at a Glance

- dev branch → developer creates a feature branch → develop locally, deploy to DEV workspace via CLI.
- Feature branch complete → open pull request → peer review → merge into develop.
- Merge into develop → GitHub Actions "Draft a release" workflow → deploy to QA, run tests, code coverage, security scans → create release branch → open PR into main.
- Release PR merged/closed → GitHub Actions deploys bundle to PROD → creates Release Tag → opens PR to merge main back into dev.
- Critical production issue → hotfix branch off main → fix → merge back into both main and dev.

Reference: Atlassian — Comparing Git workflows: Gitflow Workflow —
https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow

Section 10 — Overview of Git with Databricks (Preview)

Your slide deck introduces the next topic here — Git fundamentals — as a lead-in to the following chapter. It is included below for completeness, since the lecture begins covering it before the material provided to build this guide ends.

10.1 What Is Git?

Git is a free and open-source software framework designed to track changes in source code during software development.

10.2 Git Benefits

Git enables teams to collaborate on code by tracking changes, facilitating rollback and recovery, and enabling multiple developers to work in parallel and integrate changes efficiently. A distributed workflow ensures each developer has a full local repository, enhancing flexibility and reliability. Git is optimized for high-performance handling of large projects, and its security features include cryptographic integrity checks to prevent data corruption.

10.3 Git Tools & Services

Category	Examples / Description
Hosting platforms	GitHub, GitLab, Bitbucket, Azure DevOps — cloud-based repositories with CI/CD, issue tracking, and team collaboration.
GUI / CLI integration	SourceTree, GitKraken, VS Code Git Integration — provides different interfaces for managing repositories.
CI/CD integration	Automated testing and deployment pipelines.
Code review & collaboration	Pull requests and merge approvals streamline teamwork.
Security & access control	Role-based permissions and audit logs enhance repository security.

The slides note that upcoming material (in the next chapter) will cover Git operations, seamless integration for using remote Git repos while developing code inside Databricks notebooks, and CI/CD capabilities — the Repos REST API enables integration of data and AI projects into CI/CD pipelines, allowing users to automate Git workflows.

Glossary — Key Terms Explained

This glossary pulls together every important term from Chapter 8 in one place, in plain language, for quick lookup during revision.

DevOps
A culture and set of practices combining software engineering with IT operations, aimed at automating and streamlining the build-test-release-operate cycle so software reaches users faster and more reliably.

DataOps
DevOps principles applied specifically to data engineering — automating, testing, and monitoring data pipelines to keep data flowing reliably from source to consumption.

MLOps
DevOps principles applied specifically to machine learning — automating model training, deployment, and monitoring so ML models move from development into production reliably and stay accurate over time.

CI (Continuous Integration)
The practice of frequently merging code from multiple contributors into a shared repository, with automated builds and tests running on every commit, so problems are caught early rather than at the end of a project.

Continuous Delivery vs. Continuous Deployment
Both are forms of "CD" that automate getting tested code into staging/production, but Continuous Delivery stops one step short of production and waits for a human to click "deploy," while Continuous Deployment removes that manual gate and releases to production automatically once tests pass.

YAML ("YAML Ain't Markup Language")
A human-readable data format that uses indentation (rather than brackets or tags like JSON/XML) to represent structured configuration. Databricks Asset Bundles are configured entirely in YAML (the databricks.yml file).

Infrastructure as Code (IaC)
The practice of defining and managing infrastructure (clusters, jobs, permissions, cloud resources, etc.) using version-controlled configuration files rather than manual point-and-click setup. IaC makes environments reproducible, auditable, and easy to promote between dev/stage/prod. DABs and Terraform are both examples of IaC tools for Databricks.

Terraform
Terraform is a popular open-source Infrastructure as Code tool built by HashiCorp. Instead of clicking through a cloud console or workspace UI to create resources, you describe the desired end-state of your infrastructure in configuration files (written in HashiCorp Configuration Language, .tf files), and Terraform figures out what needs to be created, changed, or destroyed to reach that state — a plan/apply workflow (terraform plan, then terraform apply). The Databricks Terraform Provider is a plugin that teaches Terraform how to talk to the Databricks REST API, so you can provision and manage almost any Databricks object (workspaces, clusters, jobs, Unity Catalog objects, permissions, and more) as code. In your Chapter 8 slides, Terraform is presented as one of three ways to orchestrate a CI/CD deployment (alongside the UI and the raw REST API/SDK) — it is described as very powerful and the tool of choice for platform administrators managing infrastructure at scale, but with a steeper learning curve for data engineers who don't already have an infrastructure background. In practice, many organizations use Terraform to

provision the underlying workspace/infrastructure once, and then use DABs on top of that for day-to-day application/pipeline deployment — the two tools are complementary, not mutually exclusive.

Declarative Automation Bundles (DABs) / Databricks Asset Bundles

Databricks' own YAML-based Infrastructure-as-Code tool, purpose-built for Databricks projects. A bundle bundles together your code (notebooks, Python files), your resource definitions (jobs, pipelines), and your environment configuration (dev/stage/prod) into one deployable, version-controlled unit, driven by the databricks.yml file and the databricks bundle CLI commands. "Databricks Asset Bundles" is the older name; "Declarative Automation Bundles" is the current name in Databricks documentation — they refer to the same feature.

databricks.yml

The single required configuration file at the root of every bundle project. It defines the bundle's name, its resources (jobs, pipelines, etc.), its deployment targets (dev/stage/prod), and optionally variables, workspace settings, permissions, artifacts, includes, and sync paths.

Target (in a DAB)

A named deployment environment (e.g. development, staging, production) defined under the targets mapping in databricks.yml, each with its own workspace URL, mode, and configuration overrides. You deploy to a specific target using databricks bundle deploy -t <target-name>.

Databricks CLI

A command-line tool that wraps the Databricks REST API, letting you manage and automate Databricks resources (clusters, jobs, bundles, workspace files, and more) from a terminal, script, or CI/CD pipeline instead of clicking through the web UI.

Databricks SDK

A set of official software development kits (available for Python, Java, Go, and R) that let you call Databricks REST API operations directly from your own application code, rather than making raw HTTP requests.

REST API

"Representational State Transfer" Application Programming Interface — the underlying web-based interface (HTTP requests/responses) that the Databricks CLI, SDK, and Terraform provider all ultimately call to create, read, update, or delete resources in a Databricks workspace.

Databricks Connect (v2)

A Python client library that lets you write and run Spark code on your local machine (e.g. inside VS Code) while the actual execution happens remotely on a Databricks cluster — giving you local IDE features (debugging, autocomplete) with remote compute power.

Databricks Extension for VS Code

An official Visual Studio Code extension that brings Databricks project management, bundle deployment/validation, and Databricks Connect debugging directly into the VS Code interface.

Unity Catalog

Databricks' unified data governance layer, providing a three-level namespace (catalog.schema.table) for organizing and securing data. In a CI/CD context, using a separate catalog per environment (dev/stage/prod) is a common way to isolate data between environments.

pytest

A widely used Python testing framework known for its simple test-writing syntax (functions prefixed with test_), built-in assert-based checks, automatic test discovery, and a large plugin ecosystem. It is commonly used to write unit tests for PySpark functions.

Testing Pyramid

A model for structuring your test suite as three layers — many fast, cheap unit tests at the base; fewer, slower integration tests in the middle; and a small number of slow, expensive, end-to-end system tests at the top.

Pipeline Expectations

Built-in data quality rules you can attach to a Lakeflow Declarative Pipeline (e.g. “this column must not be null”) that validate records as they flow between pipeline stages, and can drop, warn on, or fail the pipeline for records that violate them.

Service Principal

A non-human, application-level identity used to authenticate automated processes (like a CI/CD pipeline) to Databricks, instead of using a real user’s personal login — improves security and auditability for automated deployments.

PAT (Personal Access Token)

A long-lived token tied to a user or service principal account that can be used in place of a password to authenticate CLI/API/Terraform requests to Databricks.

GitHub Actions

GitHub’s built-in CI/CD automation feature. You define workflows as YAML files in a repository’s .github/workflows folder, and GitHub runs them automatically in response to events like pull requests or pushes — commonly used to run databricks bundle validate / deploy / run steps automatically.

GitFlow

A structured Git branching model that uses a long-lived main branch (production) and develop branch (integration), plus short-lived feature, release, and hotfix branches, to organize how code moves from individual development through review and into production.

Feature Branch / Pull Request (PR) / Hotfix Branch

A feature branch is a short-lived branch created off develop for building one feature or fix in isolation. A pull request (PR) is a request to merge one branch into another, typically requiring peer review before merging. A hotfix branch is a special branch created directly off main to urgently patch a production issue, later merged back into both main and develop.

Quick Revision — Exam Cheat Sheet

A fast, last-minute pass over the highest-yield facts from this chapter.

Must-remember distinctions

- Continuous Delivery = automated up to staging, manual click to deploy to production. Continuous Deployment = fully automatic all the way to production.
- DataOps = DevOps for data pipelines. MLOps = DevOps for ML models.
- Unit tests (fastest/cheapest, base of pyramid) → Integration tests (middle) → System tests (slowest/costliest, top of pyramid).
- Three ways to orchestrate CI/CD on Databricks: Manually (UI), Programmatically (REST API/SDK), Terraform. DABs sit on top of the CLI as the recommended, Databricks-native IaC approach.
- "Databricks Asset Bundles" and "Declarative Automation Bundles" (DABs) are the same feature; current docs use the newer name.

databricks.yml essentials

- Must be named exactly databricks.yml, located at the bundle root, and there is only one per bundle (though it can include other files via the include mapping).
- Required top-level mapping: bundle (must include a name).
- Other top-level mappings: resources, targets, variables, workspace, permissions, artifacts, include, sync.
- Reference a variable with ${var.<name>}; reference a default substitution with ${bundle.name}, ${bundle.target}, ${workspace.file_path}, etc.
- A variable is type string by default; use type: complex for structured/multi-field values.
- A lookup variable resolves to an object's ID (e.g. cluster ID) by looking up its name — supported object types include alert, cluster, cluster_policy, dashboard, instance_pool, job, metastore, notification_destination, pipeline, query, service_principal, warehouse.
- Target-level variables mapping is how you override a variable's value per environment (dev vs. prod) — the variable must still have a default defined at the top level.

CLI command sequence

- databricks bundle validate → checks your bundle config for errors.
- databricks bundle deploy -t <target> → deploys bundle resources to the named target (defaults to the default: true target if -t is omitted).
- databricks bundle run -t <target> <resource_key> → runs a specific resource by its resource key (not its display name).

Default bundle templates

- default-python — Python projects.
- default-sql — SQL projects.
- dbt-sql — DBT projects.

- mlops-stacks — ML operations stacks.
- Custom templates need at minimum: databricks_template_schema.json + databricks.yml.tmpl.

GitFlow with DABs — sequence to remember

1. Feature branch off dev → deploy locally to DEV via CLI.
2. PR + peer review → merge into develop.
3. Merge triggers GitHub Actions → deploy to QA, run tests/coverage/security scans → create release branch → open PR into main.
4. Release PR closed → GitHub Actions deploys to PROD, creates a Release Tag, opens PR to merge main back into dev.
5. Critical prod bug → hotfix branch off main → fix → merge into both main and dev.

Practice Questions

Scenario-style questions in the spirit of the certification exam. Try to answer before checking the explanation.

Q1. A team wants changes to be automatically pushed to a staging environment on every commit, but wants a person to explicitly approve before anything reaches production. Which practice does this describe?

A. Continuous Integration only
B. Continuous Deployment
C. Continuous Delivery
D. Continuous Monitoring

Answer: C. Continuous Delivery automates the pipeline up to staging/pre-production, but keeps a manual approval gate before production. Continuous Deployment would remove that manual gate entirely.

Q2. Which top-level mapping is required in every databricks.yml file?

A. resources
B. targets
C. bundle
D. variables

Answer: C. The bundle mapping (with at least a name) is the only mapping that must be present in every bundle configuration file.

Q3. You need a variable whose value should differ between development and production. Where do you define the override?

A. Inside the resources mapping
B. Inside the targets mapping, under a target-specific variables block
C. In a separate .env file
D. Inside the artifacts mapping

Answer: B. Overrides are defined per-target using a variables block nested under that target in the targets mapping. The variable must still have a default value declared at the top-level variables mapping.

Q4. Which type of test sits at the base of the testing pyramid and should make up the majority of your automated tests?

A. System tests
B. Integration tests
C. Unit tests
D. Smoke tests

Answer: C. Unit tests are fast, cheap, and provide the broadest coverage, so they form the base (and bulk) of the testing pyramid.

Q5. Which CLI command would you run to check a bundle for configuration errors before deploying it?

A. databricks bundle run
B. databricks bundle deploy
C. databricks bundle validate
D. databricks bundle init

Answer: C. databricks bundle validate reviews the bundle's configuration for accuracy issues without deploying anything.

Q6. A variable is declared without a type key in databricks.yml. What type does Databricks assume by default?

A. complex

B. list

C. string

D. integer

Answer: C. Unless explicitly declared as type: complex (or another type), a custom variable is assumed to be of type string.

Q7. Which of the following is NOT one of the three orchestration approaches discussed for deploying Databricks projects?

A. Manually via the UI

B. Programmatically via REST API/SDK

C. Using the Databricks Terraform provider

D. Using Kubernetes Helm charts

Answer: D. The three approaches covered are manual (UI), programmatic (REST API/SDK), and Terraform. Helm charts / Kubernetes were not part of this lecture's orchestration options.

Q8. In the GitFlow + DABs pattern described in this chapter, what happens immediately after a Release PR is closed?

A. A hotfix branch is automatically created

B. GitHub Actions deploys the bundle to PROD, creates a Release Tag, and opens a PR to merge main back into dev

C. The feature branch is deleted with no further action

D. Unit tests are skipped for the next release

Answer: B. Closing the Release PR triggers the production deployment workflow: deploy to PROD, tag the release, and open a PR to sync main back into dev.

Additional Resources & Reference Links

Official, current Databricks (and related) documentation for everything covered in this chapter — useful for going deeper than the slides, or for hands-on practice.

Declarative Automation Bundles (DABs)

What are Declarative Automation Bundles? (overview) — https://docs.databricks.com/aws/en/dev-tools/bundles/

Bundle configuration syntax (databricks.yml) — https://docs.databricks.com/aws/en/dev-tools/bundles/settings

Full configuration reference (all top-level mapping keys) — https://docs.databricks.com/aws/en/dev-tools/bundles/reference

Declarative Automation Bundles resources reference — https://docs.databricks.com/aws/en/dev-tools/bundles/resources

Bundle project templates (default & custom) — https://docs.databricks.com/aws/en/dev-tools/bundles/templates

Databricks CLI

Databricks CLI overview — https://docs.databricks.com/aws/en/dev-tools/cli/

Install or update the Databricks CLI — https://docs.databricks.com/aws/en/dev-tools/cli/install

Databricks CLI tutorial (install + first commands) — https://docs.databricks.com/aws/en/dev-tools/cli/tutorial

Terraform

Databricks Terraform provider (getting started guide) — https://docs.databricks.com/aws/en/dev-tools/terraform/

Databricks Terraform provider docs (Terraform Registry) — https://registry.terraform.io/providers/databricks/databricks/latest/docs

Local development (VS Code)

Databricks extension for Visual Studio Code (overview) — https://docs.databricks.com/aws/en/dev-tools/vscode-ext/

Install the Databricks extension for VS Code — https://docs.databricks.com/aws/en/dev-tools/vscode-ext/install

Testing

pytest official documentation — https://docs.pytest.org/

CI/CD & GitHub Actions

CI/CD on Databricks (overview of the full workflow) — https://docs.databricks.com/aws/en/dev-tools/ci-cd/

GitHub Actions for Databricks (official examples) — https://docs.databricks.com/aws/en/dev-tools/ci-cd/github

Best practices for operational excellence — https://docs.databricks.com/aws/en/lakehouse-architecture/operational-excellence/best-practices

Git & GitFlow

Atlassian — Comparing Git workflows: Gitflow Workflow — https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow

Exam tip: Documentation URLs and product names (e.g. “Asset Bundles” → “Declarative Automation Bundles”) can change as Databricks updates its docs — if a link above looks outdated by the time you review this, search docs.databricks.com for the page title instead.